AnaToPrint
Administrative Manual

Prepared by The Slice Is Right

Collaborators: Bryan S, Cooper S, Ethan W, Greg M, Israk A

Client: Terry Yoo

COS 497 - Computer Science Capstone 1

April 15th, 2026

Version 1.0.0

AnaToPrint
Administrator Manual

Table of Contents

Page

1. Introduction

This section introduces the AnaToPrint Administrator Manual, explains its intended audience and scope, and provides reference documentation for further reading. Administrators new to the system should read this section before proceeding to installation or deployment procedures.

1.1 Purpose of This Document

This Administrator Manual provides the information necessary to install, deploy, maintain, and support *AnaToPrint*, a client-side web application that converts DICOM-based medical CT scans into 3D-printable STL and G-code outputs. This document is intended for system administrators, technical support personnel, and future project maintainers who may not be familiar with the system but are responsible for ensuring its continued operation and availability. Because AnaToPrint operates entirely on the client side, it does not rely on a backend server, database, or cloud infrastructure. As a result, administrative responsibilities focus on:

- Building and configuring the application from source
- Hosting or distributing static application files
- Maintaining the Node.js development environment
- Managing and updating project dependencies
- Verifying system functionality through testing
- Assisting users with operational and technical issues

This manual begins with an overview of the system and its requirements, followed by detailed procedures for installation, deployment, and maintenance. It concludes with troubleshooting guidance covering common errors, failures, and known system limitations.

References

[1] I. Arafat, G. Michaud, C. Stepankiw, B. Sturdivant, and E. Wyman, *Critical Design Review Document*, 2026.
[2] I. Arafat, G. Michaud, C. Stepankiw, B. Sturdivant, and E. Wyman, *Software Requirements Specification (SRS)*, 2026.
[3] I. Arafat, G. Michaud, C. Stepankiw, B. Sturdivant, and E. Wyman, *System Design Document (SDD)*, 2026.
[4] I. Arafat, G. Michaud, C. Stepankiw, B. Sturdivant, and E. Wyman, *User Interface Design Document (UIDD)*, 2026.
[5] S. Mitchell, *Administrator Manual Template*, adapted for COS 497: Capstone II, University of Maine, 2026.

2. System Overview

This section provides a high-level overview of AnaToPrint, including its purpose, architecture, and the hardware and software requirements for both the build environment and end users. Understanding the system's design and constraints is essential context for the installation and maintenance procedures described in later sections.

2.1 Background

AnaToPrint is a client-side web application developed as part of the Laboratory for Convergent Science at the University of Maine. It accepts a medical CT scan in DICOM format and produces files suitable for 3D printing: 1. A polygonal STL mesh of anatomical structures (bone, skin, or muscle), or 2. A GCode file that, when printed, mimics the X-ray attenuation properties of the original tissue. All processing, including DICOM parsing, 3D rendering, segmentation, and file export, occurs entirely within the user's web browser. No data is transmitted to any server, and no file leaves the user's machine unless explicitly exported by the user. User medical data will be protected and private.

The role of the system administrator for AnaToPrint is narrow compared to a traditional server-backed application. There is no database to maintain, no backend to monitor, and no user data to back up. The administrator's primary responsibilities are: building the application from source using the provided Node.js toolchain, making the resulting static files available to users (via a local web server or institutional file host), keeping dependencies up to date over time, and running the automated test suite to verify correctness after any changes. The administrator should be comfortable working in a terminal, running npm commands, and have a basic understanding of how static web applications are structured.

2.2 Hardware and Software Requirements

Development and Build Environment

The following are required on the machine used to build and maintain AnaToPrint:

Component	Requirement
Operating System	Windows 10+, macOS 12+, or Ubuntu 22.04+ (or equivalent Linux distribution)
CPU	Any modern x86-64 or ARM64 processor
RAM	8 GB minimum; 16 GB recommended
Disk Space	~2 GB free for dependencies, build artifacts, and Playwright browser binaries
Node.js	V18 LTS or newer
npm	V9 or newer (included with Node.js)

Table 2.1 - Hardware and Software Requirements

End-User Environment

No software installation is required by end users. The recommended minimum for reliable performance with large DICOM series is:

Component	Requirement
Browser	Chrome 110+, Firefox 110+, or Safari 16.4+
RAM	8 GB (large CT volumes may consume 2-4 GB of browser memory)
OS	Windows 10+, macOS 12+, or a modern Linux Desktop
GPU	Integrated graphics acceptable; dedicated GPU improvesd 3D rendering performance

Table 2.2 - Recommended Performance Requirements

3. Administrative Procedures

This section covers all key administrative tasks for setting up, maintaining, and supporting AnaToPrint. It is intended for system administrators and is written to be accessible regardless of technical background. Each subsection addresses a specific area of administration.

Note: AnaToPrint is currently installed by cloning the project's Github repository. The team is actively working on packaging it as a simple downloadable executable to make installation easier in the future. These instructions reflect the current process.

3.1 Installation

This section walks you through how to install and run AnaToPrint on your local machine. Before you begin, make sure the following software is installed on your computer:

- Git - used to download the project files from GitHub
- Node.js and npm - used to install dependencies and run the application.
- A supported web browser - Google Chrome, Mozilla Firefox, or Apple Safari. Microsoft Edge should also work but is not officially tested or supported.

If you are unsure if you have these whether you have installed, open a terminal (or command prompt on Windows) and type:

```
git --version
node --version
npm --version
```

Each command should display a version number. If you see an error message instead, visit the links below to download and install the required software:

Download Git
Download Node.js (npm is included)

Step 1. Clone the repository

```
git clone https://github.com/Team-Ochre-Capstone/AnaToPrint
cd AnaToPrint
```

Step 2. Open terminal and install dependencies:

```
npm install
```

Step 3. Start development server:

```
npm run dev
```

Step 4. Once the server starts, open browser and navigate to http://localhost:5173
After opening this you should see the AnaToPrint interface. The application is now running and ready to use.

Troubleshooting Installation Issues:

- **Make sure Node.js is installed and up to date. Try running npm install again. If the issue persists, delete the node_modules folder and try again.**
- **Make sure the development server started successfully in step 3. Check the terminal for any error messages.**
- **Git may not be installed or may not be added to your system's PATH. Reinstall Git from the link above and restart your terminal.**

3.2 Routine Tasks

The following tasks should be performed on a regular basis to keep AnaToPrint running smoothly and avoid compatibility or installation issues.

Verify Required software is installed:
Before using AnaToPrint, confirm that the following tools are available on your system:

- npm (Node Package Manager) - Required to install dependencies and run the server.
- Git - required to pull updates from the project repository.

You can check their installation by running the following:

```
git --version
node --version
```

Both commands should return a version number. If either is missing, refer to Section 3.1 for download links.

Pull the Latest Updates:
When the development team releases changes or fixes, you will need to update your local copy of the project. To do this, open a terminal in the AnaToPrint folder and run:

```
git pull
```

After pulling updates, reinstall dependencies in case anything changed:

```
npm install
```

Note:
It is a good habit to run git pull and npm install each time you plan to use the application, especially if it has been a while since your last session.

3.3 Periodic Administration

The following tasks should be performed periodically for example, once a month or before a major demonstration or deployment to keep the system healthy and up to date.

Update Git:
An outdated version of Git may cause issues when pulling updates or cloning repositories. To check and update Git visit:

https://git-scm.com/downloads

Download the latest version for your operating system and run the installer.

Update Node.js and npm:
Keeping Node.js and npm up to date helps ensure compatibility with the project's dependencies. To check your current versions:

```
node --version
npm --version
```

To update npm specifically, run:

```
npm install -g npm@latest
```

For Node.js, download the latest LTS release from:
https://nodejs.org/en/download

Reinstall Dependencies After Updates:
After updating Node.js or npm it is good practice to reinstall the project's dependencies:

```
cd AnaToPrint
npm install
```

Verify the Application Still Runs:
After performing any updates, confirm that the application launches correctly by running:

```
npm run dev
```

Then open your browser to http://localhost:5173 and verify the interface loads as expected.

3.4 User Support

If you encounter an issue that you are unable to resolve using the troubleshooting steps in this manual, the AnaToPrint development team is available to assist. Please reach out by email and include as much detail as possible about the problem you are experiencing, including any error messages shown in your terminal or browser.

Team Contact Emails:
- bryan.sturdivant@maine.edu
- israk.ayasin@maine.edu
- gregory.michaud@maine.edu
- cooper.stepankiw@maine.edu
- ethan.wyman@maine.edu

When contacting support, please try to include the following information:
- A description of what you were trying to do when the issue occurred.
- Any error messages displayed in the terminal or browser.
- The operating system you are using (e.g., Windows 11, macOS Ventura, Ubuntu 22.04).
- The versions of Node.js, npm, and Git on your system (see section 3.2 for how to check).

3.5 Verifying the Installation

AnaToPrint includes an automated test suite that can be used to confirm the application is functioning correctly after installation or after pulling updates. There are two types of tests: unit and component tests, which check individual parts of the code, and end-to-end tests, which simulate a real user interacting with the application in a browser.

Unit & Component Tests (Vitest):
These tests check that the internal logic of the application is working correctly. To run them, open a terminal, navigate into the webapp folder, and run:

```
cd webapp
npx vitest run
```

To also generate a coverage report showing how much of the code is tested, run:

```
npx vitest run --coverage
```

A passing result will show all tests marked as passed in the terminal output. If any tests fail, note the error messages and refer to Section 3.4 for support contact information.

End-to-End Tests (Playwright):

These tests simulate a real user opening the application in a browser and performing actions. They verify that the full workflow from uploading a CT scan to exporting a file behaves as expected.

The first time you run end-to-end tests, you need to install the required browsers. Run this once:

```
npx playwright install --with-deps
```

Then, to run the full end-to-end test suite across all supported browsers:

```
npm run test:e2e
```

Note: End-to-end tests require the development server to not already be running on port 5173. If you have 'npm run dev' running in another terminal, stop it first before running Playwright tests.

Troubleshooting Test Failures:

- **Try deleting the node_modules folder, running npm install again, and re-running the tests.**
- **Re-run 'npx playwright install --with-deps' to ensure all browser binaries are installed.**
- **Confirm the development server is running with 'npm run dev' and that your browser is navigating to http://localhost:5173.**

3.6 Verifying the Installation

Understanding the layout of the project files can help administrators locate documentation, troubleshoot issues, or find relevant files when working with the development team. The AnaToPrint repository is organized as follows:

- Contains project documentation, including the original project proposal and Software Requirements Specification (SRS). This is a good starting point for understanding what the application is designed to do.
- Contains the full React frontend application. This is where all of the application source code lives, including the user interface, DICOM processing logic, and 3D visualization components.
- Contains the end-to-end test files used by Playwright. These tests simulate user workflows and verify the application behaves correctly from a browser perspective.

Note:
Administrators do not need to modify any files in these folders under normal circumstances. This overview is provided for reference only, in case you need to share specific files with the development team when reporting an issue.

4. Troubleshooting

In this section, we outline how to handle error messages, system failures, and other issues that may arise during normal use. You'll find practical tips for responding to serious errors, along with guidance on interpreting what those messages mean and where they can be found in the code. We also describe the system's known bugs and limitations, including where they occur, why they exist, and how they may affect user or administrator tasks.

Errors:

ERROR\ISSUE	PLACE IN CODE	DESCRIPTION
"No DICOM files found"	webapp\src\hooks\useDicomUpload.ts	Triggered when the user uploads a file or folder but no valid .dcm / DICOM-formatted files can be detected within the selection. The upload process is halted and the user is notified to verify they selected the correct files.
"Failed to generate PNG zip for G-code export."	webapp\src\pages\ExportPage.tsx	Occurs during G-code export when the system is unable to produce the accompanying PNG slice archive (zip). This may result from a rendering failure, memory constraint, or missing slice data at the time of export.
"Error parsing DICOM file:"	webapp\src\utils\dicomUtils.ts	Thrown when a DICOM file cannot be read or interpreted correctly, typically due to a malformed header, unsupported transfer syntax, missing mandatory tags, or file corruption. The specific file and error details are appended to the message.
"Could not save to session storage:"	webapp\src\utils\storage.ts	Raised when an attempt to write data to sessionStorage fails, commonly caused by exceeding the browser's storage quota, operating in a private/incognito context with restricted storage, or a browser security policy blocking access.
"Could not read from session storage:"	webapp\src\utils\storage.ts	Raised when an attempt to retrieve data from sessionStorage fails, typically due to the key not existing, a browser security restriction, or the session having been cleared or expired before the read was attempted.

Table 4.1 - Troubleshooting Errors

Limitations and Bugs

Limitation	Where It Occurs	Why It Exists	Affect on User
Maximum of 5GB File Sizes	Import Page	This prevents the user from breaking the application by sending too much information at once.	They might be limited to the amount of data they can send at once.
Machine Hardware Limitations	Everywhere	This is a baseline “requirement” for the software so that we can be certain the hardware will support it.	The user's machine must be able to run Windows 11, and either Edge, Chrome, Firefox, or Safari. Especially with larger volumes of data, it might take longer depending on the hardware. Please see Section 2.2 for specific requirements.

Table 4.2 - Limitations and Bugs

Appendix A – Team Review Sign-off

This document confirms that all undersigned members of the AnaToPrint have thoroughly reviewed the entirety of this Administrative Manual for the Enabling 3D Printing of a Medical CT-Scan system. By signing below, each team member acknowledges their understanding of the requirements and formally agrees that the content, scope, and structure of this document are accurate and complete, providing a suitable foundation for the subsequent phases of the project. The comment section provided for each team member is to be used for noting any minor suggestions, editorial feedback, or non-substantive points of clarification.

NAME	SIGNATURE	DATE	COMMENTS
Israk Arafat		3/25/26	
Gregory Michaud		3/25/26	
Cooper Stepankiw		3/25/26	
Bryan Sturdivant		3/25/26	
Ethan Wyman		3/25/26	

Appendix B – Document Contributions

Member Name	Primary Responsibilities	Estimated Percentage
Israk Arafat	Section 4	20%
Gregory Michaud	Section 1, Table of Contents	20%
Cooper Stepankiw	Appendix, Section 4	20%
Bryan Sturdivant	Section 2	20%
Ethan Wyman	Section 3	20%
Total		100%